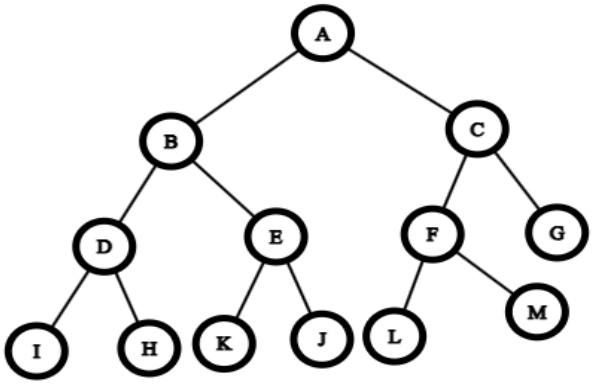
Andrei Martinez

Assignment 1
CS 4630- Intro Artificial Intelligence

Total: 50 points

Question 1: (10 points) You are given a tree structure with the root node **A** and goal node **M**. Perform both **Breadth-First Search (BFS)** and **Depth-First Search (DFS)** to reach the goal node. Use the table below to record your steps for each algorithm.

For each row:
Node Visited: Indicate the node that has just been expanded.
List: Show the current contents of the queue (for BFS) or stack (for DFS) **after** the expansion.
Note: Always expand nodes from **left to right** as they appear in the tree.
Goal Node: M



Solution:

Node Visited	List
A	B,C
B	C,D,E
C	D,E,F,G
D	E,F,G,I,H
E	F,G,I,H,K,J
F	G,I,H,K,J,L,M
G	I,H,K,J,L,M
I	H,K,J,L,M
H	K,J,L,M
K	J,L,M
J	L,M
L	M
M	

Node Visited	List
A	A
B	B,C
D	I,H,E,C
I	H,E,C
H	E,C
E	K,J,C
K	J,C
J	C
C	F,G
F	L,M,G
L	M,G
M	G

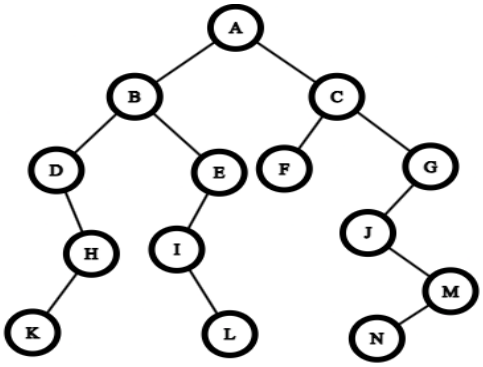
Question 2: (10 points) You are given the following tree structure for a search problem. Your task is to **perform Iterative Deepening Search (IDS)** to find the goal node **J**, starting from the root node **A**.

Each iteration should expand the search frontier one level deeper. At each depth cutoff level c, perform a Depth-Limited Search (DLS) and record the nodes visited during that iteration. Continue the process until the goal node J is found.

Instructions: For each depth cutoff c, fill in the table below with:

- The value of c (i.e., the depth limit).
- The sequence of nodes visited during that depth-limited iteration.
- The full list of visited nodes up to that point in the search.

Use the table provided below to complete your answer.



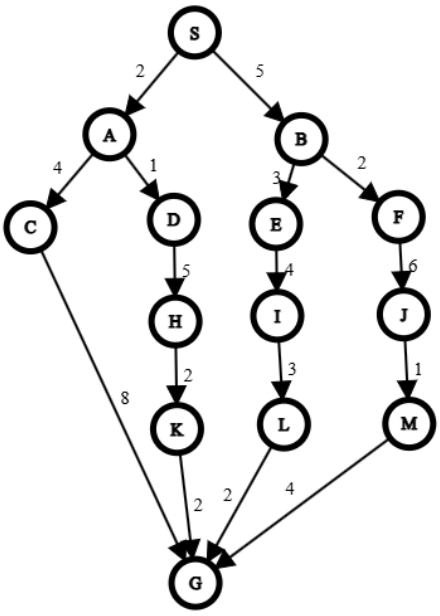
Solution:

Depth cutoff, c	Node Visited	List
0	A	A
1	A,B,C	A,B,C
2	A,B,D,E,C,F,G	A,B,C,D,E,F,G
3	A,B,D,H,E,I,C,F,G,J	A,B,C,D,E,F,G,H,I,J

Question 3: (10 points) You are given the directed graph below. Your task is to perform **Uniform-Cost Search (UCS)** to find the optimal path from the **start node S** to the **goal node G**.

What to answer:

- The completed table.
- The optimal path found (from S to G).
  S,A,D,H,K,G
- The total cost of the optimal path.
  12
- The total number of nodes expanded (including the goal node).
  11



Solution:

Expanded Node	Node List
S	S
A	S,A
D	S,A,D
B	S,B
C	S,A,C
F	S,B,F
H	S,A,D,H
E	S,B,E
K	S,A,D,H,K
I	S,B,E,I
G	S,A,D,H,K,G

Question 4: (10 points) The figure below shows a robot navigation field, where the red square (d2) is the robot, and green square (b7) is the goal. The shad squares (such as c2, d3 etc.) are obstacles. The robot is not allowed to move in diagonal line. Nodes are coded using an alphabet letter followed by a digit (such as b1, b2 etc.). When two sibling nodes are inserted into fringe (queue), use deque order to favor node with a lower alphabet and a lower digit. For example, if d1 and e2 are sibling nodes, d1 will be dequeued first (because "d" has a lower alphabetic order than "e"). If e1 and e2 are sibling nodes, e1 will be dequeued first (because "1" has a lower digit than "2"). **Node expanded/visited does not need to be revisited.**

Summary:

- **Start:** red square **d2**.
- **Goal:** green square **b7**.
- **Moves:** 4-connected (Up, Down, Left, Right only — no diagonals).
- **Obstacles:** all shaded cells (e.g., **c2, c3, d3, d4, d5, d6, d8, b5, b6, b8**).
- **Cost:** each move costs 1.

- **Transition cost (g):** the number of steps taken from the start node to the current node.
- **Heuristic (h):** Manhattan distance from the current node to the goal:

$$h(n) = |\Delta row| + |\Delta col|$$

- **Evaluation function (f):**

$$f(n)=g(n)+h(n)$$

- **Tie-breaking when inserting siblings into the fringe (priority queue by increasing f=g+h)**:
  1. prefer the **lower alphabet** (b < c < d < e), then
  2. for the same letter, prefer the **lower digit** (…1 < …2 < …3).

e1	e2	e3	e4	e5	e6	e7	e8	e9
d1	d2	d3	d4	d5	d6	d7	d8	d9
c1	c2	c3	c4	c5	c6	c7	c8	c9
b1	b2	b3	b4	b5	b6	b7	b8	b9

Tasks

1. Apply **A*** search algorithm from **d2** to **b7**.
2. For each expansion, list the node expanded and the neighbors generated with their (g,h,f).
3. Report: path found, path cost, and the set (count) of expanded nodes.

Solution:

Fringe: f(N)=h(N)+g(N)	**Node visited/expanded**
0.D2, 0,7,7	D2
1.D1, 1,8,9. E2 1,8,9	D1
2.C1, 2,7,9. E2 1,8,9. E1 2,9,11	C1
3.B1, 3,6,9. E2 1,8,9. E1 2,9,11	B1
4.B2, 4,5,9. E2 1,8,9. E1 2,9,11	B2
5.B3, 5,4,9. E2 1,8,9. E1 2,9,11	B3
6.B4, 6,3,9. E2 1,8,9. E1 2,9,11	B4
7.C4, 7,4,11. E2 1,8,9. E1 2,9,11	E2
8.E3, 2,7,9. E1 2,9,11. C4 7,4,11	E3
9. E4, 3,6,9. E1 2,9,11. C4 7,4,11	E4
10. E5, 4,5,9. E1 2,9,11. C4 7,4,11	E5
11. E6 5,4,9. E1 2,9,11. C4 7,4,11	E6
12. E7 6,3,9. E1 2,9,11. C4 7,4,11	E7
13. D7 7,2,9. E1 2,9,11. C4 7,4,11	D7
14. C7 8,1,9. E1 2,9,11. C4 7,4,11	C7
15. B7 9,0,9. E1 2,9,11. C4 7,4,11	B7

Final path: d2,e2,e3,e4,e5,e6,e7,d7,c7,b7

Path cost:9 steps

Expanded nodes (count): 15

d2,d1,c1,b1,b2,b3,b4,e2,e3,e4,e5,e6,e7,d7,c7,b7

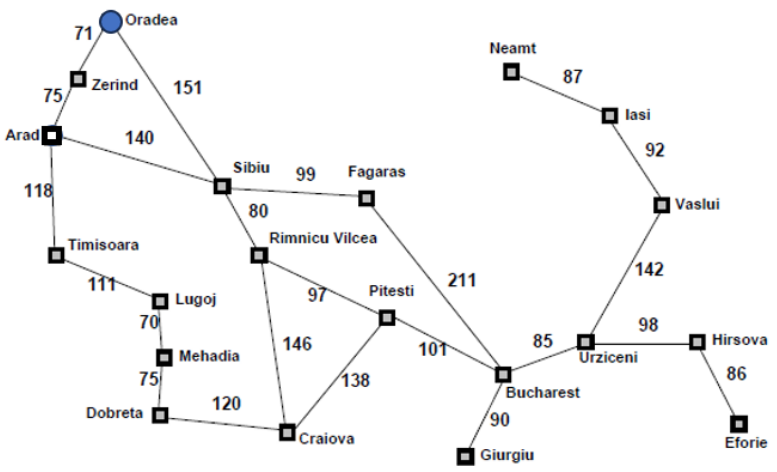
Question 5: (10 points) You are given a graph that represents cities and the road distances between them. Each node represents a city, and the edges between the cities are labeled with the road distances. Additionally, the table on the right provides the straight-line (heuristic) distance from each city to the goal city, **Bucharest**.

Task:
Perform **Greedy Best-First Search** on the given graph to find a path from the **start node Oradea** to the **goal node Bucharest**.

1. Clearly show the steps taken, including the node expansion order.
2. Use only h(n) to guide your path selection, where:
   - h(n) = straight-line distance to Bucharest from the current node (as provided)
3. Indicate the final path found by each algorithm and the total cost of the path.

Deliverables:

- Search tree expansion steps for the algorithm
- Final path and cost for Greedy Best-First Search



H(n)

Straight−line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Solution:

Using Greedy BFS, it'll always choose the fastest path, as it is "greedy." We start at Oradea.

Oradea -> Zerind (h=374)/Sibiu(h=253)

Sibiu -> Oradea(h=380)/Arad(h=366)/Fagaras(h=178)/Rimnicu Vilcea(h=193) -- unoptimal.

Fagaras -> Sibiu(h=253)/Bucharest(h=0)

The final path is Oradea -> Sibiu -> Fagaras -> Bucharest. The total path costs 461.

This path is unoptimal, as instead of choosing Fagaras, we can go down Rimnicu Vilcea -> Pitesti -> Bucharest for a lower cost.